complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + anymatch() existence checks

filter() null checks + anymatch() existence checks

filter() comparisons + anymatch() existence checks

filter() method calls returning boolean + anymatch() existence checks

distinct() rely on equals() / hashcode() + anymatch() existence checks

distinct() immutable objects + anymatch() existence checks

map() transform objects + anymatch() existence checks

map() create new objects + anymatch() existence checks

map() call pure functions + anymatch() existence checks

flatmap() return stream + anymatch() existence checks

flatmap() split data + anymatch() existence checks

flatmap() expand collections + anymatch() existence checks

sorted() natural ordering + anymatch() existence checks

sorted() custom comparator + anymatch() existence checks

sorted() multi-field compare + anymatch() existence checks

peek() logging + anymatch() existence checks

peek() debugging + anymatch() existence checks

peek() metrics + anymatch() existence checks

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✓ for-each loops
- ✓ Iterator
- ✓ Collections / Arrays
- ✓ HashSet for distinct
- ✓ Manual sorting with Comparator
- ✓ Null checks
- ✓ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** for **each requested Stream combination**, where:

- **Terminal operation = anyMatch()**
- Purpose = **existence checks** (returns true if **any element** satisfies the predicate)
- Demonstrates **short-circuiting** (stops on first match)
- Java **8+** compatible
- No collectors / reduce / custom aggregation

---

1 filter() boolean checks + anyMatch() — Existence check

```
import java.util.*;

public class FilterBooleanAnyMatch {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(1, 3, 4, 7);
      boolean exists = false;

      for (Integer n : list) {
         if (n > 0 && n % 2 == 0) {
            exists = true;
            break;
         }
      }

      System.out.println(exists);
   }
}
```

---

2 filter() null checks + anyMatch() — Existence check

```
import java.util.*;

public class FilterNullAnyMatch {
   public static void main(String[] args) {

      List<String> list = Arrays.asList(null, "Anurag", "Amit");
      boolean exists = false;

      for (String s : list) {
         if (s != null && s.startsWith("A")) {
            exists = true;
            break;
         }
      }

      System.out.println(exists);
   }
}
```

---

3 filter() comparisons + anyMatch() — Existence check

```
import java.util.*;

public class FilterComparisonAnyMatch {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(20000, 45000, 60000);
      boolean exists = false;

      for (Integer s : list) {
         if (s >= 30000 && s > 50000) {
            exists = true;
            break;
         }
      }

      System.out.println(exists);
   }
}
```

---

4 filter() method calls returning boolean + anyMatch() — Existence check

```
import java.util.*;

public class FilterMethodAnyMatch {

   static class User {
      boolean active;
      User(boolean active) { this.active = active; }
      boolean isActive() { return active; }
   }

   public static void main(String[] args) {

      List<User> list = Arrays.asList(
            new User(false),
            new User(true)
      );

      boolean exists = false;

      for (User u : list) {
         if (u.isActive() && u.isActive()) {
            exists = true;
            break;
         }
      }

      System.out.println(exists);
   }
}
```

---

5 distinct() rely on equals() / hashCode() + anyMatch() — Existence check

```
import java.util.*;

public class DistinctEqualsAnyMatch {

   static class Person {
      String name;
      Person(String name) { this.name = name; }

      public boolean equals(Object o) {
         if (!(o instanceof Person)) return false;
         Person p = (Person) o;
         return name.equals(p.name);
      }

      public int hashCode() {
         return name.hashCode();
      }
   }

   public static void main(String[] args) {

      List<Person> list = Arrays.asList(
            new Person("Anurag"),
            new Person("Anurag"),
            new Person("Ravi")
      );

      Set<Person> distinct = new HashSet<Person>(list);

      boolean exists = false;

      for (Person p : distinct) {
         if (p.name.equals("Ravi")) {
            exists = true;
            break;
         }
      }

      System.out.println(exists);
   }
}
```

---

6 distinct() immutable objects + anyMatch() — Existence check

```
import java.util.*;

public class DistinctImmutableAnyMatch {
   public static void main(String[] args) {

      List<String> list = Arrays.asList("Java", "Java", "Python");
      Set<String> distinct = new HashSet<String>(list);

      boolean exists = false;

      for (String lang : distinct) {
         if (lang.startsWith("P")) {
            exists = true;
            break;
         }
      }

      System.out.println(exists);
   }
}
```

---

7 map() transform objects + anyMatch() — Existence check

```
import java.util.*;

public class MapTransformAnyMatch {
   public static void main(String[] args) {

      List<String> list = Arrays.asList("anurag", "amit");
      boolean exists = false;

      for (String s : list) {
         String upper = s.toUpperCase();
         if (upper.equals("AMIT")) {
            exists = true;
            break;
         }
      }

      System.out.println(exists);
   }
}
```

---

8 map() create new objects + anyMatch() — Existence check

```
import java.util.*;

public class MapCreateObjectAnyMatch {

   static class UserDTO {
      String name;
      UserDTO(String name) { this.name = name; }
   }

   public static void main(String[] args) {

      List<String> list = Arrays.asList("Anurag", "Amit");
      boolean exists = false;

      for (String s : list) {
         UserDTO dto = new UserDTO(s);
         if (dto.name.equals("Amit")) {
            exists = true;
            break;
         }
      }

      System.out.println(exists);
   }
}
```

---

9 map() call pure functions + anyMatch() — Existence check

```
import java.util.*;

public class MapPureFunctionAnyMatch {

   static int square(int n) { return n * n; }

   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(2, 3, 4);
      boolean exists = false;

      for (Integer n : list) {
         int sq = square(n);
         if (sq == 16) {
            exists = true;
            break;
         }
      }

      System.out.println(exists);
   }
}
```

---

10 flatMap() return Stream + anyMatch() — Existence check

```
import java.util.*;

public class FlatMapStreamAnyMatch {
   public static void main(String[] args) {

      List<List<Integer>> list = Arrays.asList(
            Arrays.asList(1, 2),
            Arrays.asList(3, 4)
      );

      boolean exists = false;

      for (List<Integer> inner : list) {
         for (Integer n : inner) {
            if (n == 3) {
               exists = true;
               break;
            }
         }
      }

      System.out.println(exists);
   }
}
```

---

11 flatMap() split data + anyMatch() — Existence check

```
import java.util.*;

public class FlatMapSplitAnyMatch {
   public static void main(String[] args) {

      List<String> list = Arrays.asList("Java Stream", "Flat Map");
      boolean exists = false;

      for (String s : list) {
         String[] parts = s.split(" ");
         for (int i = 0; i < parts.length; i++) {
            if (parts[i].equals("Stream")) {
               exists = true;
               break;
            }
         }
      }

      System.out.println(exists);
   }
}
```

---

12 flatMap() expand collections + anyMatch() — Existence check

```
import java.util.*;

public class FlatMapExpandAnyMatch {
   public static void main(String[] args) {

      Map<String, List<String>> map = new HashMap<String, List<String>>();
      map.put("A", Arrays.asList("1", "2"));
      map.put("B", Arrays.asList("3"));

      boolean exists = false;

      for (List<String> values : map.values()) {
         for (String v : values) {
            if (v.equals("2")) {
               exists = true;
               break;
            }
         }
      }
      System.out.println(exists);
   }
}
```

---

13 sorted() natural ordering + anyMatch() — Existence check

```
import java.util.*;

public class SortedNaturalAnyMatch {
   public static void main(String[] args) {

      List<Integer> list = new ArrayList<Integer>(Arrays.asList(5, 1, 3));
      Collections.sort(list);

      boolean exists = false;

      for (Integer n : list) {
         if (n == 1) {
            exists = true;
            break;
         }
      }

      System.out.println(exists);
   }
}
```

---

14 sorted() custom comparator + anyMatch() — Existence check

```
import java.util.*;

public class SortedCustomAnyMatch {
   public static void main(String[] args) {

      List<String> list = new ArrayList<String>(
            Arrays.asList("Anurag", "Ravi", "Amit"));

      Collections.sort(list, Collections.reverseOrder());

      boolean exists = false;

      for (String name : list) {
         if (name.equals("Amit")) {
            exists = true;
            break;
         }
      }
      System.out.println(exists);
   }
}
```

---

15 sorted() multi-field compare + anyMatch() — Existence check

```
import java.util.*;

public class SortedMultiFieldAnyMatch {

   static class Employee {
      String name;
      int age;

      Employee(String name, int age) {
         this.name = name;
         this.age = age;
      }
   }

   public static void main(String[] args) {

      List<Employee> list = new ArrayList<Employee>();
      list.add(new Employee("Anurag", 30));
      list.add(new Employee("Anurag", 25));
      list.add(new Employee("Ravi", 35));

      Collections.sort(list, new Comparator<Employee>() {
         public int compare(Employee e1, Employee e2) {
            int nameCompare = e1.name.compareTo(e2.name);
            if (nameCompare != 0) return nameCompare;
            return e1.age - e2.age;
         }
      });

      boolean exists = false;

      for (Employee e : list) {
         if (e.age == 25) {
            exists = true;
            break;
         }
      }

      System.out.println(exists);
   }
}
```

---

16 peek() logging + anyMatch() — Existence check

```
import java.util.*;

public class PeekLoggingAnyMatch {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(1, 2, 3);
      boolean exists = false;

      for (Integer n : list) {
         System.out.println("Checking: " + n);
         if (n == 2) {
            exists = true;
            break;
         }
      }

      System.out.println(exists);
   }
}
```

---

17 peek() debugging + anyMatch() — Existence check

```
import java.util.*;

public class PeekDebugAnyMatch {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(10, 20, 30);
      boolean exists = false;

      for (Integer n : list) {
         if (n == 20) {
            System.out.println("Debug breakpoint");
         }
         if (n == 20) {
            exists = true;
            break;
         }
      }

      System.out.println(exists);
   }
}
```

---

18 peek() metrics + anyMatch() — Existence check

```
import java.util.*;

public class PeekMetricsAnyMatch {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(5, 10, 15);
      int processed = 0;
      boolean exists = false;

      for (Integer n : list) {
         processed++;
         if (n == 10) {
            exists = true;
            break;
         }
      }

      System.out.println("Exists: " + exists);
      System.out.println("Processed elements: " + processed);
   }
}
```